



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Developing AI Apps and Agents on Azure (AI-103)

TGS Ref No: TGS-2023036651

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 4.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
2.0	26 Aug 2026	Prior AI-102 courseware release	Tertiary Infotech Academy Pte Ltd
3.0	27 Sep 2026	AI-103 title transition	Tertiary Infotech Academy Pte Ltd
4.0	27 Sep 2026	AI-103 course title and current Foundry scope; technical evidence model; 10 detailed self-contained labs.	Tertiary Infotech Academy Pte Ltd

TABLE OF CONTENTS

01	How to Use This Guide
02	Before You Start
03	Lab 01 - Plan, Manage, Monitor, and Secure an Azure AI Solution
04	Lab 02 - Responsible AI, Content Safety, Governance
05	Lab 03 - Generative AI, Foundry, Azure OpenAI, Foundry Workflow, RAG
06	Lab 04 - Agentic AI, Foundry Agent Service, Multi-Agent Concepts
07	Lab 05 - Image and Video Generation, Multimodal Understanding
08	Lab 06 - NLP, Language, Speech, Translation
09	Lab 07 - Generative Text Analysis and Translation
10	Lab 08 - Azure AI Search, Knowledge Mining, Vector Search
11	Lab 09 - Document Intelligence and Content Understanding
12	Lab 10 - AI-103 Capstone and Course Review
13	Quick Command Reference
14	Assessment Flow and Support

How to Use This Guide

Use the trainer deck to understand mechanisms and decisions. Use this guide for the detailed click paths, commands, code, expected results, diagnostics, and evidence required in each lab.

Authority	Current reference
Approved course	Developing AI Apps and Agents on Azure (AI-103) · TGS-2023036651 · 2 days / 16 hours
Course enquiries	https://www.tertiarycourses.com.sg/
AI-103 study guide	Skills measured from 16 Apr 2026
Microsoft course	AI-103T00-A: Develop AI apps and agents on Azure
LMS	https://lms-tms.tertiaryinfotech.com/

Before You Start

- Use the Azure subscription or lab environment supplied by the trainer.
- Install current Azure CLI/Python tooling only when the lab requires it.
- Authenticate with Microsoft Entra ID where supported; never save live keys in the repository.
- Use synthetic data and remove training resources after evidence has been captured.
- For every lab, retain request IDs, deployment/model versions, screenshots or JSON outputs, and a short interpretation.

Lab 01 - Plan, Manage, Monitor, and Secure an Azure AI Solution

Source folder: labs/lab-01-plan-manage-secure-azure-ai-solution

Objectives

- Select appropriate Azure AI services.
- Plan resource deployment and endpoints.
- Identify authentication, keys, monitoring, and cost controls.
- Prepare an AI solution architecture.

Scenario

A company wants a customer support AI platform with document search, answer generation, sentiment analysis, speech transcription, and image processing. You must plan the Azure AI resources.

Steps

1. Map Requirements to Services

2. Plan Foundry Resources

Document:

Hub or project name
Region
Model deployment
Default endpoint
Connected data source
Responsible AI settings

3. Plan Security

Record:

- Role-based access control.

- Managed identities.
- Key protection.
- Private endpoint or network controls.
- Logging and diagnostics.
- Secrets storage.

4. Plan Monitoring and Cost

Document:

Token or transaction usage
Latency
Errors
Content safety events
Budget alerts
Diagnostic logs

5. Draw Architecture

Use diagrams.net:

Application -> API layer -> Foundry/Azure AI services -> Search index/storage -> Monitoring

6. AI-103 Deployment and Operations Decision

For one Foundry app and one agent, record the model/deployment choice, project connection, managed identity, private-network decision, quota, and CI/CD promotion path. Define a release record with a pinned model version, an evaluation dataset, a rollback target, and owners for cost and safety alerts. Use a synthetic architecture if the trainer has not provided an Azure subscription.

Validation

You should have a service map, security plan, monitoring plan, and architecture diagram.

Checkpoint Questions

1. Why is service selection part of AI engineering?
2. Why should keys be protected?
3. What should be monitored in an AI service?
4. What is the role of responsible AI planning?

Course Focus

AI engineering starts with selecting the right service, securing it, monitoring it, and planning for cost and responsible use.

Acceptance Evidence

Evidence pack: resource topology, RBAC assignment, private connectivity decision, successful SDK call, and an alert driven by a real metric.

Troubleshooting Checklist

5. Capture the full error and request/operation ID before retrying.
6. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
7. Validate payload/schema fields and input content type.
8. Check quota, rate limits, network resolution, private connectivity, and service status.
9. Record the corrective action and rerun the acceptance check.

Lab 02 - Responsible AI, Content Safety, Governance

Source folder: labs/lab-02-responsible-ai-content-safety-governance

Objectives

- Apply responsible AI principles.
- Configure content safety concepts.
- Explain content filters, blocklists, prompt shields, and harm detection.
- Design a responsible AI governance framework.

Scenario

The support AI assistant may receive abusive user input, produce inaccurate answers, or expose sensitive data. You must design responsible AI controls.

Steps

1. Review Responsible AI Principles

Write one control for:

Fairness
Reliability and safety
Privacy and security
Inclusiveness
Transparency
Accountability

2. Define Content Safety Controls

Document:

- Harm categories.
- Severity levels.
- Content filters.
- Blocklists.
- Prompt shields.
- Groundedness checks.
- Human escalation rules.

3. Create a Safety Policy

Write:

Allowed use:
Disallowed use:
Blocked content:
Human review trigger:
Audit log requirement:
Escalation owner:
Review frequency:

4. Test Governance Scenarios

Decide what should happen when:

10. User asks for harmful instructions.
11. Model produces low-confidence answer.
12. Prompt contains personal data.
13. Retrieved document is outdated.

14. User asks for legal advice.

5. Create an Incident Plan

Define how to report, investigate, remediate, and prevent repeat AI safety incidents.

6. Evaluate an Agent Safeguard

Add a tool call that could change a customer ticket. Define a risk tier, allowed tool arguments, human approval threshold, prompt-injection test, audit trace, and kill switch. Test one permitted case and one denied case using a trainer-provided synthetic trace or a live training agent. Mark synthetic traces as simulations. Explain which safety metric blocks release even if quality and latency pass.

Validation

You should have responsible AI controls, safety policy, scenario decisions, and incident plan.

Checkpoint Questions

15. What are content filters?
16. What is a prompt shield?
17. Why is human escalation important?
18. Who is accountable for AI output?

Course Focus

Responsible AI is implemented through policy, configuration, monitoring, human review, and governance.

Acceptance Evidence

Evidence pack: risk register, test dataset, per-category confusion matrix, filter configuration, exception workflow, and incident rehearsal result.

Troubleshooting Checklist

19. Capture the full error and request/operation ID before retrying.
20. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
21. Validate payload/schema fields and input content type.
22. Check quota, rate limits, network resolution, private connectivity, and service status.
23. Record the corrective action and rerun the acceptance check.

Lab 03 - Generative AI, Foundry, Azure OpenAI, Foundry Workflow, RAG

Source folder: labs/lab-03-generative-ai-foundry-openai-prompt-flow-rag

Objectives

- Plan a generative AI solution with Microsoft Foundry.
- Select and deploy a model conceptually.
- Design prompt templates, Foundry SDK workflows, and retrieval-augmented generation.
- Implement a RAG pattern conceptually.
- Evaluate model and flow outputs.

Scenario

The company wants a grounded assistant that answers questions from internal support articles and generates draft replies.

Steps

1. Select a Model

Record:

Model name
Use case fit
Context window
Latency
Cost
Safety settings
Deployment option

2. Design Prompt Templates

Create templates for:

- 24. Direct answer.
- 25. Answer with sources.
- 26. Escalation when context is missing.

3. Plan Foundry Workflow

Draw:

User question -> classify intent -> retrieve context -> prompt template -> model -> safety check -> response

4. Design RAG

Document:

- Source documents.
- Cleaning.
- Chunking.
- Embeddings.
- Vector store.
- Retrieval filters.
- Citation format.
- Evaluation method.

5. Evaluate Outputs

Score:

Validation

You should have model selection notes, prompt templates, tool-augmented workflow diagram, RAG design, and evaluation table.

Checkpoint Questions

- 27. What is RAG?
- 28. Why are prompt templates useful?
- 29. What does tracing help debug?
- 30. How can feedback improve a generative AI solution?

Course Focus

Generative AI solutions need grounded data, evaluation, safety settings, monitoring, and feedback loops.

Acceptance Evidence

Evidence pack: chunk sample, index schema, hybrid query trace, cited answer, groundedness score, latency, and a negative test with no supporting evidence.

Troubleshooting Checklist

31. Capture the full error and request/operation ID before retrying.
32. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
33. Validate payload/schema fields and input content type.
34. Check quota, rate limits, network resolution, private connectivity, and service status.
35. Record the corrective action and rerun the acceptance check.

Lab 04 - Agentic AI, Foundry Agent Service, Multi-Agent Concepts

Source folder: labs/lab-04-agentic-ai-foundry-agent-service

Objectives

- Explain agent roles and use cases.
- Plan Foundry Agent Service resources.
- Design custom agent tools and workflows.
- Review multi-agent orchestration concepts.

Scenario

The support AI assistant needs to look up order status, search support documents, draft a reply, and escalate complex cases to a human agent.

Steps

1. Define Agent Use Case

Write:

Agent goal:
User types:
Allowed tools:
Disallowed actions:
Human escalation trigger:
Success metric:

2. Plan Agent Resources

Document:

- Foundry project.
- Model deployment.
- Search tool.
- Function or API tools.
- Authentication.
- Logging and trace settings.

3. Design Tool Calls

Create a table:

4. Design Orchestration

Draw:

User request -> agent plan -> tool call -> observation -> model reasoning -> response -> feedback

5. Multi-Agent Review

Explain when separate agents might be useful for retrieval, billing, technical support, and escalation review.

6. Capture a Bounded Agent Trace

Define a typed search_knowledge(query, tenant_id) tool and a typed create_ticket(summary, priority) tool. Inspect data/example-traces.json, then create your own allowed and denied traces with goal, chosen tool, validated arguments, tool result, cited evidence, approval decision, and final response. Run python3 validate.py <your-traces.json> and save the PASS output. Repeat with a cross-tenant search or unapproved ticket creation; the trace must show denial. Compare a single-agent implementation with a specialist handoff, including state partition, turn budget, latency, and failure ownership. The local checker is a synthetic control test. Use real Foundry traces only when an approved training deployment is available.

Validation

You should have agent use case, resource plan, tool table, orchestration diagram, and multi-agent notes.

Checkpoint Questions

36. What is an AI agent?
37. Why do agents need tool permissions?
38. What is orchestration?
39. Why should autonomous actions be constrained?

Course Focus

Agentic solutions combine models, tools, workflows, monitoring, safety controls, and human escalation.

Acceptance Evidence

Evidence pack: agent definition/version, tool schemas, least-privilege identity, approval gate, trace showing tool call/result, budget exhaustion test, and handoff test.

Troubleshooting Checklist

40. Capture the full error and request/operation ID before retrying.
41. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
42. Validate payload/schema fields and input content type.
43. Check quota, rate limits, network resolution, private connectivity, and service status.
44. Record the corrective action and rerun the acceptance check.

Lab 05 - Image and Video Generation, Multimodal Understanding

Source folder: labs/lab-05-image-video-generation-multimodal

Course: Developing AI Apps and Agents on Azure (AI-103) · Domain: Computer vision (10–15%)

Objectives

- Choose image generation, video generation, image editing, or multimodal understanding for a product need.

- Evaluate generated media against the source brief, safety rules, accessibility, and visual evidence.
- Define a production workflow with model version, moderation, human approval, and provenance.

Scenario

A retailer wants a synthetic product campaign image and accessible descriptions. Its support agent must also answer questions about a customer-submitted product photo. Work only with the synthetic brief and review records in this folder.

Materials

- `data/product-brief.json`: product attributes, permitted claims, and brand rules.
- `data/sample-output.json`: simulated model output for a repeatable review. It is **not** evidence of a live Foundry call.
- Python 3 standard library for the local check.
- Optional: an approved Microsoft Foundry project with an image model and a multimodal model available in the training region.

Steps

1. Select the workflow

Read `data/product-brief.json`. Write a decision table with four rows: image generation, image editing, video generation, and visual question answering. For each, state the input, output, model capability, safety gate, and whether this scenario needs it. Distinguish creating new media from interpreting existing media.

2. Design generation evidence

Draft an image prompt that uses only the approved product attributes. Record the model/deployment and version, prompt, safety policy, and expected output format. Define a rejection rule for an unsupported performance claim, missing product attribute, incorrect logo, or unsafe content. If video is proposed, add storyboard, duration, reference-media rights, watermark, and frame-level review controls.

3. Review a synthetic response

Open `data/sample-output.json`. Compare its caption, alt text, answer, and claims with the brief. Create `review.json` containing:

```
{"model_version": "sample-only", "approved": false, "unsupported_claims":
[], "missing_attributes": [], "alt_text_issue": "", "human_reviewer": "training reviewer"}
```

Populate the arrays and decision from the evidence. Run `python3 validate.py review.json`; it checks the review schema and the required decision rationale. Save the validation output.

4. Optional live Foundry extension

If the trainer has supplied approved deployments, generate a synthetic product concept image, then ask a multimodal model for a caption, accessible alt text, and an answer grounded in that image. Record the model and deployment versions, prompt, response, safety result, latency, and request ID. Compare the live result with the same brief. Label it live only when the service call and response are captured. Do not upload real customer images or publish generated media without the human gate.

5. Architecture and failure review

Draw the path: brief or uploaded media → content validation → model deployment → moderation → visual/claim evaluator → human review → approved asset or rejection. Add the indirect prompt-

injection risk from text embedded in an image. State a control for each failure and a rollback path for a changed model version.

Validation

A complete submission has the decision table, prompt and model record, review.json, passing local validation, architecture, and one rejected output with a cited reason. Live Foundry evidence is optional and must be labeled separately.

Troubleshooting

- Missing model in region: use the provided sample; record the deployment constraint.
- JSON check fails: compare required keys and value types in `validate.py`.
- Model answer invents a property: reject it and quote the unsupported claim in the review.
- Visual content may be unsafe: stop distribution and route to human review.

Cleanup

Remove only resources created in your own lab resource group after the trainer confirms evidence is saved. Keep the synthetic review files.

Reference

[Microsoft AI-103 study guide — computer vision](<https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/ai-103#implement-computer-vision-solutions-1015>)

Acceptance Evidence

Evidence pack: task selection, source brief, model/version, prompt, generated or sample output, caption and alt text, claim/safety review, and human publication decision.

Troubleshooting Checklist

45. Capture the full error and request/operation ID before retrying.
46. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
47. Validate payload/schema fields and input content type.
48. Check quota, rate limits, network resolution, private connectivity, and service status.
49. Record the corrective action and rerun the acceptance check.

Lab 06 - NLP, Language, Speech, Translation

Source folder: labs/lab-06-nlp-language-speech-translation

Objectives

- Analyze and translate text.
- Extract key phrases, entities, PII, language, and sentiment.
- Process speech with speech-to-text and text-to-speech.
- Plan translation and SSML use.

Scenario

The support platform must analyze customer messages, detect PII, translate text, transcribe calls, and generate spoken responses.

Steps

1. Map Language Tasks

2. Design Text Analysis Pipeline

Input text -> language detection -> PII detection -> sentiment -> entities -> key phrases
-> route case

3. Design Speech Pipeline

Audio input -> speech to text -> intent or keyword recognition -> response -> text to speech

4. SSML Review

Explain how SSML can control voice, pitch, rate, pauses, and pronunciation.

5. Translation Review

Document:

- Text translation.
- Document translation.
- Speech-to-text translation.
- Speech-to-speech translation.
- Human review for critical messages.

6. AI-103 Speech as an Agent Modality

For a synthetic spoken customer request, trace audio input → speech-to-text → text analysis → agent tool → text-to-speech response. Record the locale, transcript confidence or recognition reason, sensitive-content redaction, domain-term translation check, fallback for NoMatch, and human escalation point. If a live Speech resource is unavailable, label the trace as a design simulation rather than a service result.

Validation

You should have language task mapping, text pipeline, speech pipeline, SSML notes, and translation notes.

Checkpoint Questions

50. What is entity recognition?
51. Why detect PII before storing text?
52. What is SSML?
53. When should translation outputs be reviewed?

Course Focus

Language and speech solutions combine analysis, privacy controls, translation, and application integration.

Acceptance Evidence

Evidence pack: multilingual samples, PII span/redaction, recognition reasons, SSML output, word-error-rate sample, and human review for domain terms.

Troubleshooting Checklist

54. Capture the full error and request/operation ID before retrying.
55. Confirm the endpoint, API/model version, region, identity, and RBAC scope.

- 56. Validate payload/schema fields and input content type.
- 57. Check quota, rate limits, network resolution, private connectivity, and service status.
- 58. Record the corrective action and rerun the acceptance check.

Lab 07 - Generative Text Analysis and Translation

Source folder: labs/lab-07-generative-text-analysis

Course: Developing AI Apps and Agents on Azure (AI-103) · Domain: Text analysis (10–15%)

Objectives

- Extract entities, topic, sentiment, summary, and structured JSON from customer text.
- Detect sensitive content and distinguish source evidence from generated claims.
- Compare Azure Translator with an LLM translation flow for a support workflow.

Scenario

A service team receives multilingual synthetic messages. It needs a typed triage record for a support agent, with human review when the message is ambiguous or contains sensitive data.

Materials

- `data/messages.json`: three synthetic messages with expected review signals.
- `data/sample-analysis.json`: simulated model output for a repeatable local exercise; **not** a live Azure result.
- Python 3 standard library for validation.
- Optional: trainer-approved Microsoft Foundry text model and Azure Translator in the training region.

Steps

1. Define the output contract

Read the three messages. Design a JSON record with `message_id`, `topic`, `sentiment`, `entities`, `summary`, `sensitive_content`, `evidence_span`, `language`, and `review_required`. Define allowed values and a no-evidence rule. Make the evidence span an exact substring of the source message.

2. Evaluate the sample

Open `data/sample-analysis.json` and compare every record with its source. Identify any missing entity, unsupported summary, false sentiment, or sensitive identifier that should be redacted. Write the corrected records to `analysis.json` in the same folder. Run `python3 validate.py analysis.json`; fix each reported error and save the PASS output.

3. Design the Foundry prompt

Write a prompt that requests only the typed JSON fields, keeps each message separate, cites an exact evidence span, avoids ungrounded claims, and sets `review_required` when uncertain or sensitive. Record model deployment/version, temperature, input, response, request ID, and latency if the trainer provides a live deployment. Label `data/sample-analysis.json` as synthetic simulation at all times.

4. Compare translation paths

For the non-English message, compare two paths: Azure Translator followed by text analysis, and an LLM-powered translation and extraction flow. Define tests for preserved named entities, tone,

domain terms, language identification, cost, latency, and privacy. Do not silently replace the original message with a translation; retain both for audit.

5. Add agent integration

Draw the path: message → language detection → sensitive-content gate → typed analysis → schema validation → human review if needed → agent tool. State why the agent must not act on unvalidated JSON or leak identifiers into logs.

Validation

Submit analysis.json, the validator PASS output, prompt, translation comparison, and agent data-flow. Each record must have a source-bound evidence span, correct sensitive-content decision, and no invented entity. Live service evidence is optional and must be labeled separately.

Troubleshooting

- JSON parse failure: remove commentary outside the JSON object and verify types.
- Evidence span mismatch: copy the exact source substring; do not paraphrase it.
- Ambiguous sentiment: set a review rule and explain the uncertainty.
- Translation changes an entity: retain the original and route the case for review.

Cleanup

Remove only your own temporary training deployments after the trainer confirms evidence capture. Keep synthetic source and corrected records.

Reference

[Microsoft AI-103 study guide — text analysis](<https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/ai-103#implement-text-analysis-solutions-1015>)

Acceptance Evidence

Evidence pack: synthetic source messages, typed JSON, exact spans, sensitive-content decision, translation comparison, validator output, and agent routing rule.

Troubleshooting Checklist

59. Capture the full error and request/operation ID before retrying.
60. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
61. Validate payload/schema fields and input content type.
62. Check quota, rate limits, network resolution, private connectivity, and service status.
63. Record the corrective action and rerun the acceptance check.

Lab 08 - Azure AI Search, Knowledge Mining, Vector Search

Source folder: labs/lab-08-ai-search-knowledge-mining-vector-search

Objectives

- Design an Azure AI Search solution.
- Define indexes, data sources, indexers, and skillsets.
- Explain semantic and vector search.
- Plan knowledge store projections.

Scenario

The company wants searchable support documents, enriched with extracted text, entities, key phrases, and vector embeddings for RAG.

Steps

1. Design an Index

Define fields:

```
id
title
content
category
source_url
last_updated
security_label
embedding
```

2. Plan Data Sources and Indexers

Document:

- Source storage.
- Supported document formats.
- Indexer schedule.
- Change detection.
- Error handling.

3. Define a Skillset

Include:

- OCR.
- Language detection.
- Entity extraction.
- Key phrase extraction.
- Custom skill.
- Embedding generation.

4. Query Design

Write examples for:

```
Keyword search
Filter by category
Sort by last updated
Wildcard query
Semantic ranking
Vector search
Hybrid search
```

5. Knowledge Store Projections

Explain file, object, and table projections and when enriched output should be stored.

6. Ground an Agent on Retrieved Evidence

Use data/support-docs.json and data/judged-queries.json. Design one BM25 query, one vector query, and one hybrid query with a tenant filter applied before results reach an agent. Save each result ranking as JSON in the shape of data/example-rankings.json and run python3 evaluate.py

<your-rankings.json>. Record a citation to the winning chunk and a no-evidence answer. Compare relevance, p95 latency, and estimated cost for the three modes; label any invented values as illustrative. State how an indexer failure or stale source would be detected. The local scorer checks relevance and tenant isolation; it does not call Azure AI Search or measure live latency.

Validation

You should have index design, data source/indexer plan, skillset plan, query examples, and knowledge store notes.

Checkpoint Questions

- 64. What is an indexer?
- 65. What is a skillset?
- 66. What is vector search?
- 67. Why use semantic ranking?

Course Focus

Knowledge mining combines source data, enrichment, indexing, querying, and optional vector retrieval for AI applications.

Acceptance Evidence

Evidence pack: index schema, data source/indexer/skillset JSON, successful run history, judged queries, hybrid trace, security-filter negative test, and knowledge-store projection.

Troubleshooting Checklist

- 68. Capture the full error and request/operation ID before retrying.
- 69. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
- 70. Validate payload/schema fields and input content type.
- 71. Check quota, rate limits, network resolution, private connectivity, and service status.
- 72. Record the corrective action and rerun the acceptance check.

Lab 09 - Document Intelligence and Content Understanding

Source folder: labs/lab-09-document-intelligence-content-understanding

Objectives

- Use prebuilt document extraction concepts.
- Plan custom Document Intelligence models.
- Explain composed models.
- Design Content Understanding extraction workflows.

Scenario

The company processes invoices, contracts, support screenshots, audio notes, and product videos. It wants to extract structured information from multiple content types.

Steps

1. Map Document Scenarios

2. Custom Document Model Plan

Document:

Document types
Sample count
Fields to extract
Labels
Training split
Evaluation metric
Publish plan

3. Content Understanding Plan

Explain extraction from:

- Documents.
- Images.
- Video.
- Audio.

Include summarization, classification, attributes, entities, tables, and images.

4. Human Review Rules

Define confidence thresholds and when extracted data requires review.

5. Integration Flow

Draw:

```
Upload content -> extract text/entities/tables -> validate -> store structured data -> search/report
```

6. Build a Grounded Extraction Contract

Read data/maintenance-record.json and inspect data/example-extraction.json. Define an analyzer output with document_id, part_number, fault, action, confidence, and source_span or page region. Copy the example to your own JSON file, correct or change one field, and run python3 validate.py <your-file.json>. The validator requires source provenance and human review below 0.80 confidence. Trace the clean structured output into a search index or agent tool without discarding provenance. Record analyzer version, supported modality, latency and cost estimates, privacy policy, and a fallback when the field-accuracy gate fails. This local validator does not claim a live Content Understanding call.

Validation

You should have scenario mapping, custom model plan, content understanding plan, review rules, and integration flow.

Checkpoint Questions

73. When should you use a prebuilt model?
74. What is a composed model?
75. Why are confidence scores useful?
76. What content types can Content Understanding process?

Course Focus

Information extraction solutions need model selection, training data, confidence handling, and integration design.

Acceptance Evidence

Evidence pack: analyzer/model ID, sample result JSON, source polygons/spans, field-level metrics, review threshold, failure sample, and downstream search/workflow record.

Troubleshooting Checklist

77. Capture the full error and request/operation ID before retrying.
78. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
79. Validate payload/schema fields and input content type.
80. Check quota, rate limits, network resolution, private connectivity, and service status.
81. Record the corrective action and rerun the acceptance check.

Lab 10 - AI-103 Capstone and Course Review

Source folder: labs/lab-10-ai103-capstone-course-review

Objectives

- Design an end-to-end Azure AI solution.
- Map requirements to Microsoft Foundry and Azure AI services.
- Include security, monitoring, responsible AI, and cost controls.
- Build a personal review plan.
- Show how a Foundry agent uses retrieval, tools, and human approval to complete the workflow.

Scenario

You must design a production-ready customer support AI platform that supports grounded chat, document search, sentiment analysis, speech transcription, image OCR, document extraction, and human escalation.

Steps

1. Build a Service Map

2. Draw the Architecture

Include:

User application
API layer
Foundry project
Model deployment
AI Search index
Storage
Language/Speech/Vision/Document services
Monitoring
Human review queue

3. Add Governance Controls

Document:

- RBAC.
- Managed identity.
- Key protection.
- Prompt filters.
- Content safety.
- Logging and monitoring.
- Budget alerts.
- Data retention.
- Human escalation.

4. Rate Skill Confidence

Map the final design against the [AI-103 skills guide](https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/ai-103): planning and management; generative AI and agents; computer vision; text analysis; and information extraction. For each area, identify one test case, one failure case, and the evidence needed for acceptance.

5. Clean Up Resources

If you created resources:

```
az group delete --name <your-own-lab-resource-group> --yes --no-wait
```

Confirm with your instructor before deleting shared resources.

6. Create a 7-Day Review Plan

- 82. Day 1: Planning, security, monitoring, responsible AI.
- 83. Day 2: Generative AI apps, agents, and grounding.
- 84. Day 3: Agents and orchestration.
- 85. Day 4: Vision and video.
- 86. Day 5: NLP, speech, translation, question answering.
- 87. Day 6: AI Search, Document Intelligence, Content Understanding.
- 88. Day 7: Capstone review and mistakes log.

Validation

You should have a service map, architecture diagram, governance checklist, confidence matrix, cleanup plan, and review plan.

Checkpoint Questions

- 89. Which service should power grounded document search?
- 90. When should a human review AI output?
- 91. What is the difference between RAG and fine-tuning?
- 92. Which AI workload do you need to review most?

Course Focus

Azure AI engineering is about integrating the right AI services into secure, monitored, responsible, production-ready solutions.

Acceptance Evidence

Capstone evidence: architecture and data flow, ADR, threat/risk model, evaluation report, SLO dashboard, cost estimate, deployment/rollback plan, runbook, and improvement backlog.

Troubleshooting Checklist

- 93. Capture the full error and request/operation ID before retrying.
- 94. Confirm the endpoint, API/model version, region, identity, and RBAC scope.
- 95. Validate payload/schema fields and input content type.
- 96. Check quota, rate limits, network resolution, private connectivity, and service status.
- 97. Record the corrective action and rerun the acceptance check.

Quick Command Reference

Command / item	Purpose
az login --use-device-code	Authenticate Azure CLI without embedding credentials

az account show --output table	Confirm active tenant/subscription
python -m venv .venv	Create an isolated Python environment
az monitor metrics list ...	Retrieve a metric for operational evidence
git status --short	Confirm no secrets or generated private assessment files are staged

Assessment Flow and Support

98. Complete TRAQOM digital attendance.
99. Complete Assessment digital attendance.
100. Sit the 60-minute WA-SAQ, then the 60-minute PP.
101. Submit the two candidate papers on the LMS.
102. Sign the Assessment Summary Record.

Support: enquiry@tertiaryinfotech.com · +65 6100 0613 · <https://lms-tms.tertiaryinfotech.com/>